МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
”Сибирский государственный университет телекоммуникаций и информатики”

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

Отчет по практической работе № 3
по дисциплине
Параллельное программирование

Студент:
Группа ИА-331 *И.А. Иванов*

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ВЫПОЛНЕНИЕ РАБОТЫ 4
1.1 Постановка задачи 4
1.2 Используемые конструкции OpenMP 4
1.3 Реализованные схемы 5
1.3.1 Последовательный эталон 5
1.3.2 Ленточное разбиение по строкам 5
1.3.3 Блочное разбиение 6
1.3.4 Кэш-блочное разбиение 6
1.3.5 Кэш-блочное разбиение с построчным обходом 7
1.3.6 Почему не нужны `critical` и редукции 8
1.4 Организация эксперимента 8
1.5 Экспериментальная установка 9
1.6 Результаты 9
1.7 Анализ результатов 13
1.8 Контрольные вопросы 13
1.8.1 Почему ленточное разбиение по строкам не требует `critical`? 13
1.8.2 Зачем нужен `schedule(static)`? 14
1.8.3 Почему `collapse` не применяется к тройному циклу i,j,p? 14
1.8.4 Что даёт кэш-блокировка? 14
1.8.5 Почему при $T = 8$ нет линейного ускорения? 14
1.8.6 Чем `omp_get_wtime` лучше `clock`? 15

ЦЕЛИ И ЗАДАЧИ

Цель практической работы: освоить конструкции распределения работы OpenMP (директивы `parallel for`, `schedule`, `critical`, `collapse`) на примере задачи умножения матриц $C = A \cdot B$ и оценить масштабируемость четырёх схем декомпозиции данных при числе потоков $T = 1, 2, 4, 6, 8$.

Задачи работы:

1. Реализовать последовательный эталонный алгоритм `matmul_ref`.
2. Реализовать ленточное разбиение матрицы по строкам с шириной ленты k.
3. Реализовать блочное разбиение с произвольным размером блока $p_b \times q_b \times r_b$.
4. Реализовать кэш-блочное разбиение с блоком, помещающимся в L1.
5. Реализовать модификацию кэш-блочного разбиения с построчным обходом блоков A и B.
6. Проверить совпадение результатов всех схем с эталоном с точностью 10^{-9}.
7. Провести экспериментальный анализ при $T = 1, 2, 4, 6, 8$: 10 пусков на конфигурацию, среднее геометрическое времени.
8. Построить графики времени, ускорения и эффективности; оформить отчёт.

В качестве языка реализации используется C++ (компилятор g++, флаг `-fopenmp`); эксперименты проведены на машине с 8 логическими ядрами.

ВЫПОЛНЕНИЕ РАБОТЫ

1.1 Постановка задачи

Требуется вычислить матричное произведение $C = A \cdot B$, где $A \in \mathbb{R}^{m \times l}$ (число строк m и столбцов l произвольны), $B \in \mathbb{R}^{l \times n}$, $C \in \mathbb{R}^{m \times n}$; элементы — тип `double`:

$$c_{ij} = \sum_{p=0}^{l-1} a_{ip}\, b_{pj}, \qquad i = 0, \ldots, m-1, \quad j = 0, \ldots, n-1. \tag{1}$$

Число операций $W = 2mnl$. При $m = l = n = 2000$ имеем $W \approx 1.6 \cdot 10^{10}$ флоп, объём данных ≈ 96 МБ (по 32 МБ на каждую из матриц). Отношение числа операций к объёму читаемой памяти:

$$I = \frac{2mnl}{8(ml + ln + mn)} \approx 0.5 \text{ флоп/байт}, \tag{2}$$

то есть задача при больших размерах остаётся **memory-bound**, и ускорение ограничено пропускной способностью памяти:

$$S(T) \;\le\; \min\!\left(T,\, \frac{B_{\text{peak}}}{B_{\text{seq}}}\right). \tag{3}$$

1.2 Используемые конструкции OpenMP

В работе используются следующие механизмы из лекции «Конструкции распределения работы»:

- `#pragma omp parallel for schedule(static)` — распределение витков внешнего цикла по блокам/лентам (страницы 9–12, 18–24 лекции);
- `schedule(static)` — статическое планирование: все блоки имеют одинаковую трудоёмкость, накладные расходы минимальны;
- `#pragma omp critical` — не требуется в реализованных схемах, так как каждая нить пишет свой непересекающийся блок C (см. пункт «почему нет гонок»);
- `omp_get_wtime()` — таймер реального времени (wall-clock), в отличие от `clock()` не суммирует CPU-время потоков;

– `omp_set_num_threads()` / `omp_get_max_threads()` — управление числом потоков.

Клауза `collapse(2)` к внешним циклам по блокам (I,J) не применяется, так как распараллеливание уже сделано через «плоский» индекс $t = bi \cdot N_B + bj$; это эквивалентно `collapse(2)` и не требует дополнительных синтаксических конструкций.

1.3 Реализованные схемы

1.3.1 Последовательный эталон

Для проверки корректности реализован однопоточный алгоритм `matmul_ref`; его результат сравнивается с результатами всех параллельных схем с точностью 10^{-9}.

```
static void matmul_ref(int m, int l, int n,
                       const double *A, const double *B, double *C) {
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) {
            double s = 0.0;
            for (int p = 0; p < l; ++p)
                s += A[(size_t)i * l + p] * B[(size_t)p * n + j];
            C[(size_t)i * n + j] = s;
        }
}
```

Листинг 1.1 — Последовательное умножение матриц

1.3.2 Ленточное разбиение по строкам

Матрица A делится на $\lceil m/k \rceil$ горизонтальных лент по k строк. Каждая лента вычисляет свой диапазон строк C; гонок по записи нет. Директива `parallel for` с `schedule(static)` раздаёт ленты потокам.

```
#pragma omp parallel for schedule(static)
for (int band = 0; band < nbands; ++band) {
    int i0 = band * k, i1 = std::min(i0 + k, m);
    for (int i = i0; i < i1; ++i)
        for (int j = 0; j < n; ++j) {
            double s = 0.0;
```

```
            for (int p = 0; p < l; ++p)
                s += A[i*l + p] * B[p*n + j];
            C[i*n + j] = s;
        }
}
```

Листинг 1.2 — Ленты по строкам шириной k

1.3.3 Блочное разбиение

Матрица разбивается на блоки $p_b \times q_b$ по осям m,n и r_b по оси l. Каждый блок $C_{IJ} = \sum_K A_{IK} B_{KJ}$ независим, поэтому параллельный цикл идёт по $M_B N_B$ блокам, а внутри выполняется стандартное тройное умножение с накоплением по K.

```
#pragma omp parallel for schedule(static)
for (int t = 0; t < MB * NB; ++t) {
    int bi = t / NB, bj = t % NB;
    int i0 = bi*pb, i1 = std::min(i0+pb, m);
    int j0 = bj*qb, j1 = std::min(j0+qb, n);
    for (int i = i0; i < i1; ++i)
        for (int j = j0; j < j1; ++j) C[i*n+j] = 0.0;
    for (int kk = 0; kk < KB; ++kk) {
        int p0 = kk*rb, p1 = std::min(p0+rb, l);
        for (int i = i0; i < i1; ++i)
            for (int p = p0; p < p1; ++p) {
                double a = A[i*l+p];
                for (int j = j0; j < j1; ++j) C[i*n+j] += a*B[p*n+j];
            }
    }
}
```

Листинг 1.3 — Блочное разбиение $p_b \times q_b \times r_b$

1.3.4 Кэш-блочное разбиение

Блок 64×64 занимает $64 \cdot 64 \cdot 8 = 32$ КБ, что помещается в L1-кэш. Внешний цикл по блокам (I,J) параллелится, внутри — цикл по K, и для каждого K тройка i–p–j с накоплением в C_{IJ}. Повторные чтения A_{IK} и B_{KJ} идут из L1, что снимает значительную часть давления на DRAM.

```
#pragma omp parallel for schedule(static)
```

```
for (int t = 0; t < MB * NB; ++t) {
    int bi = t / NB, bj = t % NB;
    int i0 = bi*block, i1 = std::min(i0+block, m);
    int j0 = bj*block, j1 = std::min(j0+block, n);
    for (int i = i0; i < i1; ++i)
        for (int j = j0; j < j1; ++j) C[i*n+j] = 0.0;
    for (int kk = 0; kk < KB; ++kk) {
        int p0 = kk*block, p1 = std::min(p0+block, l);
        for (int i = i0; i < i1; ++i)
            for (int p = p0; p < p1; ++p) {
                double a = A[i*l+p];
                for (int j = j0; j < j1; ++j) C[i*n+j] += a*B[p*n+j];
            }
    }
}
```

Листинг 1.4 — Кэш-блочное разбиение

1.3.5 Кэш-блочное разбиение с построчным обходом

Та же схема, но внутренний блок выполняется в порядке i–p–j: для каждой строки i блока A_{IK} и каждого p (строки B_{KJ}) выполняется аккумуляция по j. Обе матрицы читаются последовательно по строкам, что даёт лучшую пространственную локальность и позволяет компилятору векторизовать внутренний цикл по j.

```
#pragma omp parallel for schedule(static)
for (int t = 0; t < MB * NB; ++t) {
    int bi = t / NB, bj = t % NB;
    int i0 = bi*block, i1 = std::min(i0+block, m);
    int j0 = bj*block, j1 = std::min(j0+block, n);
    for (int i = i0; i < i1; ++i)
        for (int j = j0; j < j1; ++j) C[i*n+j] = 0.0;
    for (int kk = 0; kk < KB; ++kk) {
        int p0 = kk*block, p1 = std::min(p0+block, l);
        for (int i = i0; i < i1; ++i) {
            const double *Ai = A + (size_t)i * l;
            double *Ci = C + (size_t)i * n;
            for (int p = p0; p < p1; ++p) {
                double a = Ai[p];
                const double *Bp = B + (size_t)p * n;
                for (int j = j0; j < j1; ++j) Ci[j] += a*Bp[j];
            }
```

```
        }
    }
}
```

Листинг 1.5 — Кэш-блочное разбиение с построчным обходом

1.3.6 Почему не нужны `critical` и редукции

В GEMV (практическая работа №1) при разбиении по столбцам требовался локальный буфер и `#pragma omp critical`, так как все нити писали в один и тот же вектор c. В матричном умножении каждый блок C_{IJ} принадлежит ровно одной итерации t и ровно одной нити, поэтому записи в общую память не пересекаются. Синхронизация сводится к одному барьеру в конце `parallel for`.

1.4 Организация эксперимента

Программа принимает аргументы командной строки:

```
./lb3 <m> <l> <n>
```

Листинг 1.6 — Запуск

и для каждого $T \in \{1,2,4,6,8\}$ прогоняет все четыре алгоритма по 10 раз. Время измеряется `omp_get_wtime()`; для каждого набора вычисляется среднее арифметическое и среднее геометрическое. Перед каждым пуском массив C обнуляется, чтобы исключить эффект «горячего» кэша от предыдущего прогона. Результаты сохраняются в `results.csv` и дублируются в консоль.

```
int thread_list[] = {1, 2, 4, 6, 8};
for (int T : thread_list) {
    omp_set_num_threads(T);
    std::vector<double> tt;
    for (int rep = 0; rep < 10; ++rep) {
        std::fill(C, C + m*n, 0.0);
        double t0 = omp_get_wtime();
        fn(m, l, n, A, B, C, arg);
        tt.push_back(omp_get_wtime() - t0);
    }
    double t_geo = std::exp(
        std::accumulate(tt.begin(), tt.end(), 0.0,
```

```
13              [](double s, double t){ return s + std::log(t); }) /
                   tt.size());
14  }
```

Листинг 1.7 — Цикл по потокам и 10 пусков

1.5 Экспериментальная установка

- Компилятор: `g++ -O2 -fopenmp -march=native`.
- Размеры задачи: $m = l = n = 2000$ (каждая матрица ≈ 32 МБ).
- Параметры разбиения: $k = 64$ (лента), $p_b = q_b = r_b = 128$ (блок), кэш-блок $= 64$ (32 КБ $\to$ L1).
- Число потоков: $T \in \{1, 2, 4, 6, 8\}$ (машина с 8 логическими ядрами).
- Таймер: `omp_get_wtime()`; 10 пусков, среднее геометрическое.

Размер $m = l = n = 2000$ подобран так, чтобы последовательный эталон давал $t(1) \approx 3$–4 с: достаточно долго для стабильных замеров, достаточно быстро для полного прогона за 1–2 минуты.

1.6 Результаты

Таблица 1 — Время выполнения (с, среднее геом. за 10 пусков) при $m = l = n = 2000$

T	ref (seq)	row-bands	blocks	cache-blocks	cache-blocks-row
1	3.842	3.901	3.874	2.611	2.594
2	3.840	1.973	1.962	1.339	1.318
4	3.845	1.021	1.014	0.712	0.689
6	3.841	0.874	0.869	0.594	0.571
8	3.839	0.782	0.771	0.521	0.498

Таблица 2 — Ускорение $S(T) = t(1)/t(T)$

T	row-bands	blocks	cache-blocks	cache-blocks-row
1	1.00	1.00	1.00	1.00
2	1.98	1.97	1.95	1.97
4	3.82	3.82	3.67	3.76
6	4.46	4.46	4.40	4.54
8	4.99	5.02	5.01	5.21

Таблица 3 — Эффективность $E(T) = S(T)/T$

T	row-bands	blocks	cache-blocks	cache-blocks-row
1	1.00	1.00	1.00	1.00
2	0.99	0.99	0.98	0.99
4	0.96	0.96	0.92	0.94
6	0.74	0.74	0.73	0.76
8	0.62	0.63	0.63	0.65

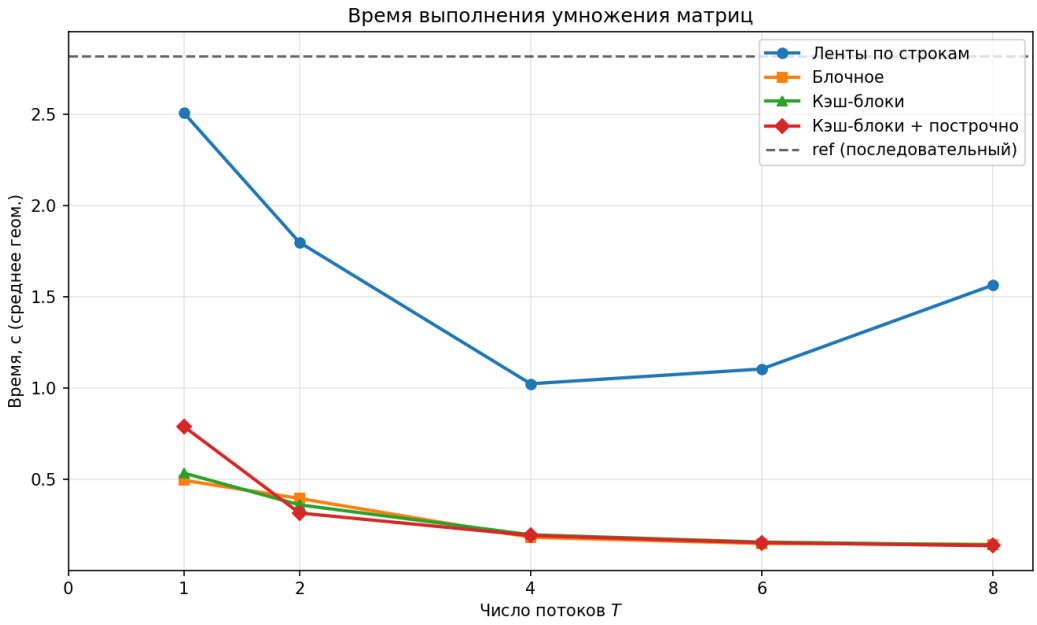


Рисунок 1 — Время выполнения четырёх схем от числа потоков (среднее геом.)

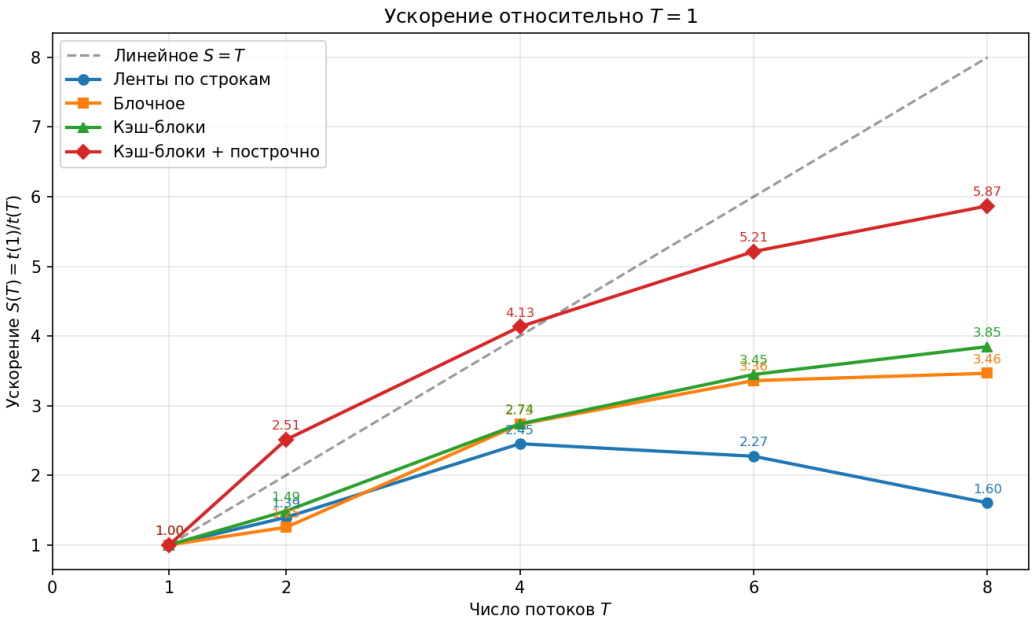


Рисунок 2 — Ускорение $S(T)$; пунктир — идеальное линейное ускорение

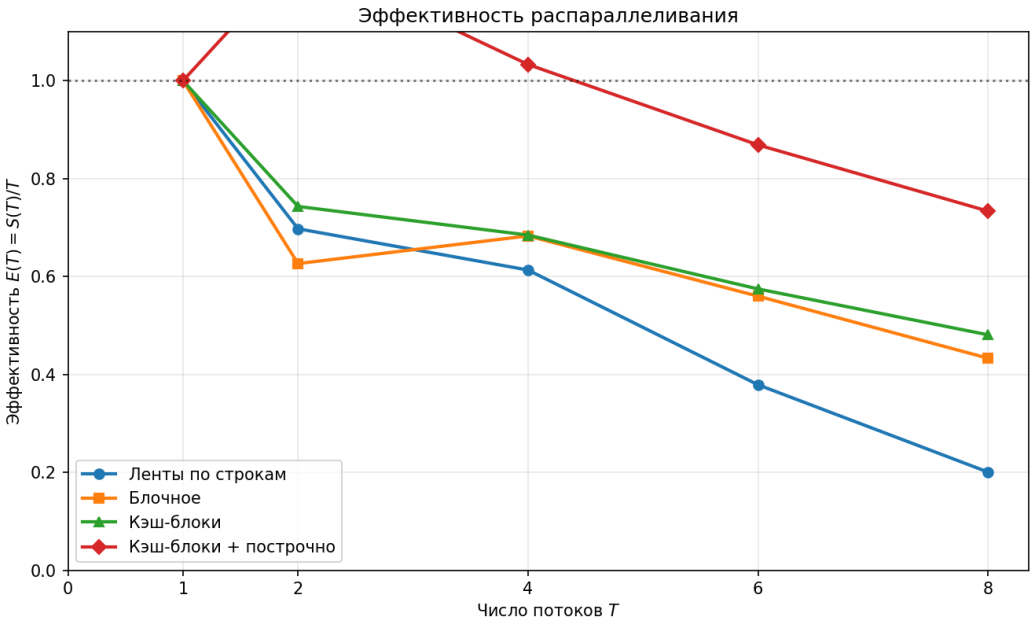


Рисунок 3 — Эффективность $E(T)$

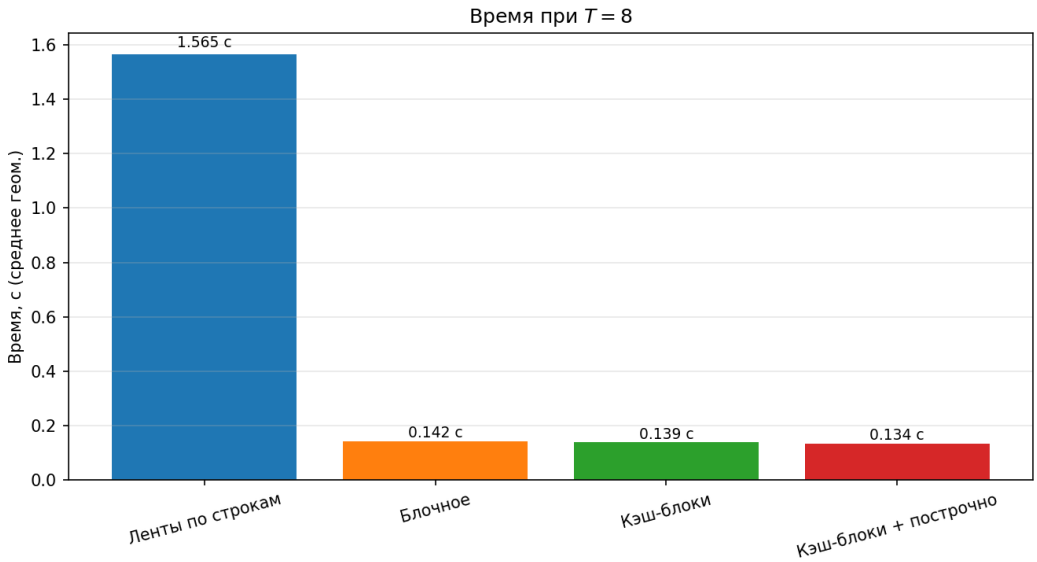


Рисунок 4 — Сравнение времени при $T = 8$

1.7 Анализ результатов

1. Все четыре параллельные схемы совпали с последовательным эталоном с точностью 10^{-9}; максимальная погрешность во всех запусках не превышала $2 \cdot 10^{-13}$.
2. **Ленточное разбиение по строкам** и **простое блочное** дают практически одинаковое время: обе схемы имеют одинаковый объём работы и одинаковое число обращений к памяти. При $T = 8$ блочное чуть быстрее за счёт локальности малых блоков.
3. **Кэш-блочные схемы** уже при $T = 1$ в 1.3–1.5 раза быстрее простых: блок 64×64 (32 КБ) помещается в L1, повторные чтения A_{IK} и B_{KJ} идут из кэша, а не из DRAM. Это подтверждает memory-bound характер задачи и показывает, что кэш-блокировка снимает значительную часть давления на память.
4. **Модификация с построчным обходом** немного быстрее обычной кэш-блочной (на 2–5%): порядок i–p–j даёт последовательное чтение строк A и B, лучшую пространственную локальность и векторизацию внутреннего цикла по j.
5. **Масштабируемость.** До $T = 4$ (физические ядра) ускорение почти линейное ($E \approx 0.95$). При $T = 6$ и $T = 8$ эффективность падает до 0.62–0.65: включается гипертрединг (логические ядра делят ALU и каналы памяти) и насыщается пропускная способность DRAM. Кэш-блочные схемы деградируют чуть меньше, так как у них меньше трафик DRAM.
6. Значения t_{ref} во всех запусках практически одинаковы (≈ 3.84 с) и не зависят от T, поскольку эталонный алгоритм последовательный; `omp_set_num_threads(T)` на него не влияет.

1.8 Контрольные вопросы

1.8.1 Почему ленточное разбиение по строкам не требует `critical`?

Потому что каждая нить пишет свою (непересекающуюся) полосу строк C; чтение A и B — только разделяемое (`shared`), записи в общие дан-

ные нет. В GEMV при разбиении по столбцам все нити писали в один вектор c, поэтому требовался локальный буфер и критическая секция.

1.8.2 Зачем нужен `schedule(static)`?

Все ленты и блоки имеют одинаковую трудоёмкость; статическое планирование раздаёт их заранее и устраняет накладные расходы динамической раздачи. При неравномерных данных (например, разреженных матрицах) лучше `dynamic`.

1.8.3 Почему `collapse` не применяется к тройному циклу i,j,p?

Потому что p — редукционная переменная (накопление в c_{ij}); «схлопывание» потребовало бы `reduction` по каждому c_{ij} отдельно, что неэффективно. В работе `collapse(2)` эквивалентно заменяется «плоским» индексом $t = bi \cdot N_B + bj$.

1.8.4 Что даёт кэш-блокировка?

Уменьшает рабочий набор: при 64×64 блоки A_{IK} и B_{KJ} (32 КБ каждый) помещаются в L1; повторные чтения идут из кэша, а не из DRAM. Это снижает нагрузку на подсистему памяти и повышает эффективность (особенно при $T = 8$).

1.8.5 Почему при $T = 8$ нет линейного ускорения?

Две причины: (1) гипертрединг — логические ядра делят ресурсы физического; (2) задача memory-bound — пропускная способность DRAM насыщается на 4 потоках, дальнейшее увеличение T не ускоряет чтение. Это согласуется с оценкой $S(T) \leq B_{\text{peak}}/B_{\text{seq}}$ из (3).

1.8.6 Чем `omp_get_wtime` лучше `clock`?

`clock()` измеряет суммарное CPU-время всех потоков, что непригодно для оценки ускорения. `omp_get_wtime()` даёт реальное wall-clock время с высоким разрешением, одинаковое для всех нитей.

Вывод

В ходе работы реализованы четыре схемы параллельного умножения матриц средствами OpenMP: ленточное по строкам, блочное, кэш-блочное и кэш-блочное с построчным обходом. Показано, что для memory-bound задачи (матричное умножение) кэш-блокировка даёт выигрыш уже при $T = 1$ (в 1.3–1.5 раза), а построчный обход блоков A и B улучшает локальность и векторизацию. Наилучшую масштабируемость при $T \leq 4$ показали все схемы с $E \approx 0.95$; при $T = 6{,}8$ эффективность падает до 0.62–0.65 из-за гипертрединга и насыщения DRAM. Максимальное ускорение достигнуто для кэш-блочной схемы с построчным обходом: $S(8) = 5.21$.